

- quality & reliability are not single phased activity, they are engineered into every SDLC activity.
- Requirement → specify quantitative reliability, availability & security requirements (to test & design targets it)
 - Design — choose architecture (redundancy) that support dependability & maintainability.
 - Implementation → follow coding standard & use version/config management to avoid regression.
 - Testing (continuous) → detect defects early & measure testing ~~etc~~ inc reliability by removing defects.
 - Deployment & Maintenance → monitoring & use reliability model and metrics to schedule fixes, monitor field failures

How is it maintained across SLOC,

- (i) Quality: It is maintained by gathering clear reqs, reviewing design, following coding standards, taking user feedback to ensure product meets expectations.
- (ii) Reliability: It is ensured by finding & fixing defects early, using stable system designs, testing under diff condition & continuous monitoring performance after deployment.

Q2) [PART II] Role of COCOMO MODEL in maintaining Quality.

the constructive cost Model ~~predicts effort~~ is a software cost estimation model that predicts effort, time & cost of a software based project based on software size (KLOC).

Proposed by Barry Boehm in 1981

COCOMO

BASIC	INTERMEDIATE	DETAILED
uses a simple eqn for overall estimation (KLOC)	include cost drivers (factor - reliability, tool experience)	Consider all project phases & cost driver. (personal factors)
Accuracy is LOW	Accuracy is Medium	Highest Accuracy.

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (E)^{d_b}$$

$E \rightarrow$ effort (in person-months)

$D \rightarrow$ Dev time (months)

a_b, b_b, c_b, d_b

const based on project type.

Phase sensitive effort multiplier

Cost driver $\rightarrow$ design & test.

Detail COCOMO

3 level product hierarchy.

$\rightarrow$ MODULE

$\rightarrow$ SUBSYSTEM

$\rightarrow$ SYS LEVEL

intermediate: →

$$f = a_i (KLOC)^{b_i} + EAF$$

$$D = c_i (E)^{d_i}$$

- (i) Product Attributes } Required software reliability
size of application db.
- ii) Hardware Attribute } Run time performance constraints & memory constraints
MEMO TAT, VM volatility
- iii) Personal Attribute } Analyst & Programmer capability
Application, VM, PL experience.
- iv) Project Attributes } Modern programming practice
use of software tools.

	ORGANIC	SEMI-DETACHED	EMBEDDED
project size	2-50 KLOC small size project	50-300 KLOC medium size project	over 300 KLOC large size project
experience	experienced developers in familiar environ.	avg previous experience on similar project	very little previous experience
example	payroll, inventory	database system, editor	ATM, air traffic control
innovation	little	Medium	significant
deadline	Not tight	Medium	Tight
environment	familiar & in-house	Medium	Complex Hardware required.

How COCOMO improves quality

- Prevent underestimation, which leads to rushed dev & more defects
- Ensure adequate staffing, avoiding high workload & burnout
- Providing early warning of high risk-module, allowing more testing
- Helps manage allocate type for quality practice s. a review & test cycle.

Q2) PART (III) Reliability Model.

Reliability model help predict failure rate & expected no. of defects as software undergoes testing.

They quantify how reliability improves over time.

PURPOSE:

Measure & predict software failure behavior

Determine how much testing is still needed.

Estimate sys reliable enough for deployment.

LOGARITHMIC POISSON MODEL. (notes).

Q2) PART (4) NEED FOR Continuous Testing Across SDLC.

Continuous testing → process of testing running automated test not only in the end, but throughout the software development process.

Goal → get quick feedback on any potential risk related to deployment of software so teams can take decision faster.

RELIABILITY MODEL : LOGARITHMIC POISSON MODEL

Software reliability growth model

MUSA OKUMOTO

describe how no. of software failure decreases over time as bugs are detected & fixed.

- proposed by musa & okumoto (1984).
- based on non-homogenous poisson process (NHPP)
- Reliability growth slows logarithmically w/ testing time.

$$\mu(t) = (\frac{1}{\theta}) \ln(1 + \theta \phi t) \quad \text{expected cumulative failure}$$

$$\lambda(t) = \phi / (1 + \theta \phi t) \quad \text{Failure intensity}$$

$\mu(t)$ = expected failure by Time t

ϕ = failure intensity decay parameter

θ = scale parameter

t = time

IT IMPROVES RELIABILITY → by predicting how many defects are left at any moment.

- * helps decide optimal release time
- * ensure software meets reliability target.

Assumption

Failure follows Poisson process

Each f Rx improves reliability

Consistent testing environment

Advantages.

capture realistic R growth

Parameters can be estimated from data.

Simple mathematical formula

Limitation.

Assume uniform fault - per time

no account for new fault introduced

may not fit complex sys.

Application.

Software testing & debugging

Reliability Prediction

Maintenance planning.

why is it importance:

- (i) Early detection: ^{prevents defect accumulation.} early detection ~~reduce the cost of fixing defect.~~
 → 1 bug found in req - avoids 50 bugs during maintenance.
- (ii) Improve reliability: Frequent testing reduce failure probability in production
- (iii) Support Agile / DEVOPS: Automated test run w every code change.
 (regression testing)
- (iv) Reduce cost: late defects are 100x more expensive, early bug detection reduced the cost of fixing defects.
- (v) Ensure performance & security: load tests & vulnerability scans performed regularly.

Q2) [PART V] Brief RISK ANALYSIS.

- 1] Security & Fraud risk: Unauthorized access / malicious activities that compromise sys / financial data.
- 2] Regulatory / compliance Risk: Failure to meet govt, legal or industry-specific financial regulations.
- 3] Availability / Reliability Risk: System downtime or failure that prevent user from accessing financial services
- 4] Performance Risk: slow processing / high latency affecting transaction & user experience.
- 5] Data Integrity Risk: Inaccurate, incomplete or ~~can~~ corrupted financial data during storage, processing

Eg

- 1] transaction integrity
- 2] PCI regulatory reporting error
- 3] downtime
- 4] high throughput /
- 5] incorrect transaction

Q) What are Key Process Areas (KPA) (CMM) give eg.

MITIGATION MEASURES:

- 1) Security: secure design (encryption in transit)
code analysis, penetration testing.
- 2) Compliance: include regulatory reqs in SAS,
schedule audit checkpoint.
- 3) Availability: redundant architecture, ~~failover~~ disaster recovery
plan, ~~also~~ chaos test.
- 4) Performance: early load / stress testing,
capacity planning from COCOMO.
- 5) Data integrity: Transaction atomicity, extensive integration
monitoring / alerting.